

◆ AWS CLOUD PRACTITIONER BOOTCAMP

STUDENT GITHUB PORTFOLIO EXPECTATIONS

Training + Proof of Work System

◆ WELCOME TO THE BOOTCAMP

This bootcamp is not only about attending sessions.

It is about building:

- ◆ technical capability
- ◆ cloud engineering discipline
- ◆ governance thinking
- ◆ architecture reasoning
- ◆ operational maturity
- ◆ and a visible proof-of-work portfolio

Your GitHub repository will become:

- ✓ your technical portfolio
- ✓ your engineering journal
- ✓ your practical proof of learning
- ✓ and your cloud capability showcase

By the end of this programme, you should not only SAY you learned AWS.

You should be able to SHOW what you built.

◆ WHY GITHUB MATTERS

In modern cloud engineering, documentation and visibility matter.

Real engineers do not only build systems.

They also:

- ✓ document systems
- ✓ explain architecture decisions
- ✓ organize projects professionally
- ✓ communicate technical thinking
- ✓ and maintain operational discipline

This is why GitHub is part of this programme.

◆ PRIMARY EXPECTATION

Every student must maintain a professional GitHub repository throughout the bootcamp.

Your repository should reflect:

- ✓ effort
 - ✓ structure
 - ✓ consistency
 - ✓ technical understanding
 - ✓ security awareness
 - ✓ and engineering growth
-

◆ EXPECTED MASTER REPOSITORY STRUCTURE

```
aws-cloud-practitioner-bootcamp/
├── README.md
├── Week-1-Foundations/
│   ├── cloud-basics.md
│   ├── aws-global-infrastructure.md
│   ├── shared-responsibility.md
│   ├── risk-scenarios.md
│   ├── iam-basics.md
│   ├── security-best-practices.md
│   └── screenshots/
├── Week-2-Compute/
│   ├── ec2-overview.md
│   ├── compute-decision-guide.md
│   └── screenshots/
├── Week-2-Storage/
│   ├── s3-notes.md
│   ├── storage-strategy.md
│   └── screenshots/
├── Week-2-Databases/
│   ├── database-comparison.md
│   └── use-case-mapping.md
├── Week-3-Networking/
│   ├── vpc-basics.md
│   ├── subnet-design.md
│   ├── network-security.md
│   └── attack-scenarios.md
```

```
├── Week-3-Architecture/
│   ├── architecture-diagrams.md
│   └── screenshots/
├── Week-4-Billing/
│   ├── pricing-models.md
│   └── cost-optimization.md
└── Week-4-Exam/
    ├── exam-strategy.md
    └── common-traps.md
```

◆ README EXPECTATIONS

Every repository MUST contain a proper:

`README.md`

Your README file is extremely important.

It explains:

- ✓ what you built
- ✓ why you built it
- ✓ how the system works
- ✓ what AWS services were used
- ✓ and what you learned



◆ MINIMUM README REQUIREMENTS

Your README should include:

- ✓ Project Title
- ✓ Project Objective
- ✓ AWS Services Used
- ✓ Architecture Explanation
- ✓ Security Considerations
- ✓ Screenshots
- ✓ Challenges Faced
- ✓ Lessons Learned

◆ EXAMPLE README STRUCTURE

```
# AWS EC2 Static Website Deployment
```

Objective
Deploy a static website using Amazon EC2 and Nginx.

AWS Services Used

- Amazon EC2
- Security Groups
- IAM
- Elastic IP

Architecture
User → Internet → Security Group → EC2 → Nginx

Security Considerations

- Restricted SSH access
- Least privilege IAM access
- Controlled inbound traffic

Key Learnings

- Linux administration
- SSH connectivity
- Web server deployment
- Infrastructure security awareness

◆ SCREENSHOTS & EVIDENCE

You must provide proof of practical work.

Expected screenshots include:

- ✓ EC2 dashboard
- ✓ Running instances
- ✓ S3 buckets
- ✓ Lambda console
- ✓ IAM configurations
- ✓ Security Groups
- ✓ Browser testing
- ✓ Terminal outputs
- ✓ CloudWatch logs

◆ EXPECTED FOLDER STRUCTURE

/screenshots
/docs
/scripts
/terraform

◆ INFRASTRUCTURE THINKING

Do not only write:

✗ “I launched an EC2 instance.”

Instead explain:

- ✓ why you selected the service
 - ✓ security implications
 - ✓ cost considerations
 - ✓ scalability considerations
-

◆ EXAMPLE OF STRONG ENGINEERING THINKING

Weak explanation:

“I used EC2.”

Strong explanation:

“I selected a t3.micro EC2 instance because it is cost-efficient for lightweight workloads and aligned with Free Tier eligibility.”

◆ COMMIT HISTORY EXPECTATIONS

Your commit history must show progression.

Good examples:

```
Initial EC2 deployment
Configured Security Groups
Added architecture documentation
Restricted SSH access
Updated S3 security notes
```

Bad examples:

```
update
files
work
new changes
```

◆ SECURITY EXPECTATIONS

You must NEVER upload:

- ✗ AWS Access Keys
 - ✗ Secret Credentials
 - ✗ Passwords
 - ✗ .pem files
 - ✗ Sensitive configurations
-

◆ GOVERNANCE HABITS

As future cloud engineers, you must begin developing governance discipline.

You should:

- ✓ organize repositories properly
 - ✓ use structured naming conventions
 - ✓ separate environments
 - ✓ avoid exposing sensitive information
 - ✓ maintain clean documentation
-

◆ ARCHITECTURE DIAGRAM EXPECTATIONS

You are encouraged to include diagrams such as:

- ✓ VPC layouts
- ✓ EC2 architecture
- ✓ S3 storage flow
- ✓ Lambda triggers
- ✓ Networking structures

Recommended tools:

- ◆ Draw.io
 - ◆ Lucidchart
 - ◆ Excalidraw
-

◆ REFLECTION & TECHNICAL WRITING

Strong students explain:

- ✓ what failed
- ✓ how they fixed issues
- ✓ what they optimized
- ✓ what they would improve later

This is how engineers become architects.

◆ WHAT STRONG STUDENTS DO

Strong students:

- ✓ explain concepts in their own words
 - ✓ document professionally
 - ✓ show technical curiosity
 - ✓ discuss security risks
 - ✓ explain architecture decisions
 - ✓ maintain clean repositories
-

◆ WHAT WEAK STUDENTS DO

Weak students:

- ✗ copy notes without understanding
 - ✗ upload random screenshots
 - ✗ avoid documentation
 - ✗ provide shallow explanations
 - ✗ show no architecture thinking
-

◆ FINAL EXPECTATION

By the end of this bootcamp, your GitHub should become:

- ✓ a technical portfolio
- ✓ an interview asset

- ✓ a proof-of-work platform
- ✓ and a cloud engineering capability archive

When someone visits your GitHub profile, they should immediately see:

- ✓ structure
 - ✓ consistency
 - ✓ practical experience
 - ✓ documentation quality
 - ✓ engineering discipline
 - ✓ and cloud capability
-

◆ FINAL PRINCIPLE

This programme is not designed to produce students who only watched sessions.

It is designed to produce individuals who can demonstrate:

- ◆ what they built
 - ◆ how they think
 - ◆ how they secure systems
 - ◆ and how they approach real cloud environments
-

◆ BOOTCAMP INFORMATION

MZ UNIFIED CLOUD BOOTCAMP

Global Academy Curriculum Portal

Unified Cloud Architecture

A practitioner-grade programme focused on:

- ✓ AWS Foundations
 - ✓ Cloud Security
 - ✓ Governance Thinking
 - ✓ Architecture Reasoning
 - ✓ Infrastructure Understanding
 - ✓ Professional Cloud Engineering Discipline
-

◆ INSTRUCTOR DETAILS

Mthulisi Zulu

Cloud Governance Architect

✉ green28@live.co.za

☎ +27 68 108 4189

◆ OFFICIAL BOOTCAMP PORTAL

Students should continuously monitor the curriculum portal for:

- ✓ updated slides
- ✓ GitHub labs
- ✓ session notes
- ✓ mock exams
- ✓ YouTube learning resources
- ✓ practical labs
- ✓ and additional updates

Portal:

<https://mthulisi28.github.io/mz-unified-cloud-bootcamp/#>

◆ CONTINUOUS LEARNING REMINDER

Cloud engineering cannot be mastered through passive learning.

You must:

- ✓ practice consistently
- ✓ build continuously
- ✓ document professionally
- ✓ and think critically

The students who succeed are the students who spend time:

- ◆ reading
- ◆ building
- ◆ troubleshooting

- ◆ documenting
 - ◆ and improving consistently
-

◆ CLOSING MESSAGE

Your GitHub portfolio starts today.

Treat every repository like a professional engineering asset.

Every lab you complete becomes:

- ◆ proof of your growth
- ◆ proof of your discipline
- ◆ and proof of your capability

Build carefully.

Document professionally.

Think like an engineer.

